Optimisation Implementation Report

Dispensary Management System

Schema Design · Security & RBAC · Indexing Strategy · Performance Benchmarking

Team Members

Harsh Jain	22110093
Shreyas Dharmatti	21110202
Sneha Gautam	22110255
Anushika Mishra	22110029
Kandarp Jani	22110104

Course: **CS 432 Databases**

Instructor: **Prof. Yogesh K Meena**

22nd March 2026

Abstract

This report documents the design and optimisation decisions made in building the **Dispensary Management System** database backend. It covers four areas: **(1)** schema design choices that enforce data integrity without duplication, **(2)** session-based JWT authentication and role-based access control across ten distinct roles, **(3)** the indexing strategy applied to the most frequently accessed SQL queries, and **(4)** quantitative benchmarking results showing execution-time improvements before and after composite-index application. Total query execution time improved by **43.1%** (22.48 ms → 12.80 ms) across 10 benchmark queries through targeted composite index design.

Contents

Video Demonstrations & Repository

GitHub Repository `github.com/Harsh-jain12/CS432_Track1_Assignment2`

Complete source code for both modules Module A (B+ Tree database engine) and Module B (Dispensary Management System) including SQL scripts, audit logs, and all documentation.

Module B UI Walkthrough Link

A walkthrough of the Dispensary Management System's frontend interface, demonstrating the login flow, role-based dashboard views, appointment booking, prescription management, and inventory control pages across multiple user roles.

Module B API Explanation Link

A detailed explanation of the REST API architecture, covering authentication endpoints, JWT token handling, RBAC enforcement via middleware, and live demonstration of curl commands across all nine user roles as documented in `API_DOCS_TO_RUN.md`.

1. Schema Design

The database, named `DispensaryManagement`, contains **20 tables** split into two logical groups: *core system tables* (authentication and access control) and *project-specific tables* (clinical and operational data). The design adheres to **Third Normal Form (3NF)** throughout every non-key attribute depends solely on the primary key, eliminating transitive dependencies and data duplication.

1.1 Table Inventory

Table 1: Complete table inventory with relationships

Group	Table	Purpose	Key Relationships
Auth	`UserLogin`	Credentials for all user types	EntityType + EntityID → 3 tables
Auth	`SystemRole`	Role lookup (10 roles)	Referenced by UserRoleMapping
Auth	`UserRoleMapping`	Many-to-many: user ↔ roles	FK → UserLogin, SystemRole
Auth	`SuperAdmin`	SuperAdmin profile data	EntityID in UserLogin
Clinical	`Member`	Student/Faculty/Staff profiles	Central entity
Clinical	`Doctor`	Doctor profiles + availability	FK in Appointment, Visit, Rx
Clinical	`StaffEmployee`	Nurses, Pharmacists, Admins	FK in MedicineDispense, Bill
Clinical	`DoctorSchedule`	Per-day working hours	FK → Doctor
Clinical	`MedicalHistory`	One-to-one with Member	FK → Member (CASCADE)
Clinical	`Appointment`	Booking records	FK → Member, Doctor
Clinical	`Visit`	Completed consultations	FK → Member, Doctor, Appointment
Clinical	`Prescription`	Prescription header	FK → Visit, Member, Doctor

Table 1

Group	Table	Purpose	Key Relationships
Clinical	`PrescriptionItem`	Individual medicines per prescription	FK → Prescription, Medicine
Clinical	`Medicine`	Medicine catalogue	FK in Inventory, PrescriptionItem
Clinical	`Inventory`	Stock batches	FK → Medicine, MedicalSupplier
Clinical	`MedicalSupplier`	Supplier directory	FK in Inventory
Clinical	`MedicineDispense`	Dispense records	FK → Prescription, Inventory
Clinical	`BillPayment`	Billing per visit	FK → Visit, StaffEmployee
Clinical	`EmergencyCase`	Emergency incidents	FK → Member, Doctor, Staff
Audit	`DirectDBChangeLog`	Detects unauthorised direct DB modifications	Written by MySQL triggers

1.2 Core Design Decisions

1.2.1 Unified Authentication No Credential Duplication

The most significant design decision was keeping credentials entirely out of domain tables. `Member`, `Doctor`, and `StaffEmployee` contain only profile data. A single `UserLogin` table holds all password hashes (bcrypt) and references the originating entity through a *polymorphic pair*: `EntityType` (`ENUM: Member, Doctor, Staff, SuperAdmin`) and `EntityID` (the primary key of the referenced row). This eliminates password duplication and ensures a single point of credential management.

Table 2: `UserLogin` polymorphic credential store (sample rows)

LoginID	Username	PasswordHash	EntityType	EntityID	IsActive
1	CS2021001	$2b$12$...	Member	1	TRUE
2	DOC001	$2b$12$...	Doctor	1	TRUE
3	NUR001	$2b$12$...	Staff	1	TRUE

Design note: `EntityType` is an `ENUM(Member, Doctor, Staff, SuperAdmin)`. Combined with `EntityID`, it forms a *polymorphic foreign key* pointing to the originating domain table. Passwords are stored as bcrypt hashes (cost factor 12) and are never logged or returned by any API endpoint.

1.2.2 Trigger-Driven Login Creation

Rather than requiring a separate API call to create login credentials after adding a member or doctor, MySQL `AFTER INSERT` triggers on `Member`, `Doctor`, `StaffEmployee`, and `SuperAdmin` automatically create the corresponding `UserLogin` row and assign the appropriate role. This guarantees that every entity always has exactly one login — enforced at the database level, not the application level.

1.2.3 Many-to-Many Role Assignment

`UserRoleMapping` implements a junction table between `UserLogin` and `SystemRole`, allowing one user to hold multiple roles (e.g., a Faculty member who is also an Admin). A `UNIQUE` constraint on `(UserLoginID, RoleID)` prevents duplicate role assignments. The ten defined roles span three permission tiers:

Student	*Faculty*	*Staff*	*Doctor*	*Nurse*
Pharmacist	*Admin*	*Technician*	*Support Staff*	*SuperAdmin*

1.2.4 Per-Day Doctor Scheduling

The original schema stored availability as two time columns (`AvailableFrom`, `AvailableTo`) and a free-text `WorkingDays` field (e.g. “Mon–Fri”). This was extended with a dedicated `DoctorSchedule` table taking one row per doctor per day thus enabling different hours on different days and proper day-of-week validation during appointment booking.

Table 3: `DoctorSchedule` — per-day schedule structure (sample rows for DoctorID = 1)

SchedID	DoctorID	DayOfWeek	StartTime	EndTime	IsActive
1	1	Monday	09:00:00	17:00:00	TRUE
2	1	Tuesday	09:00:00	17:00:00	TRUE
3	1	Saturday	10:00:00	13:00:00	TRUE

Constraints: `UNIQUE(DoctorID, DayOfWeek)` prevents duplicate schedule rows per doctor per day. `CHECK(EndTime > StartTime)` enforces logical time ordering at the database level.

1.2.5 Referential Integrity Strategy

Foreign key constraints are applied with carefully chosen cascade behaviours:

Table 4: Referential integrity cascade strategy

Relationship	ON DELETE	ON UPDATE	Rationale
MedicalHistory → Member	CASCADE	CASCADE	History is meaningless without the member
Appointment → Member	CASCADE	CASCADE	Appointments belong to the member
Visit → Appointment	SET NULL	CASCADE	Visit can exist without the original appointment
Prescription → Visit	CASCADE	CASCADE	Prescription is part of the visit
PrescriptionItem → Prescription	CASCADE	CASCADE	Items are owned by the prescription
Inventory → Medicine	RESTRICT	CASCADE	Cannot delete medicine with stock
MedicineDispense → Inventory	RESTRICT	CASCADE	Dispense records must not be orphaned
BillPayment → Visit	CASCADE	CASCADE	Bill is tied to the visit
UserLogin → (none)	N/A	N/A	Login kept on member delete (audit trail)

2. Security & Access Control

Security is implemented at two layers: **JWT-based session validation** (authentication) and **role-based access control** (authorisation). Every API endpoint except `/api/auth/login` and `/api/auth/set-password` requires a valid token.

2.1 Authentication Flow

The system uses stateless JWT tokens. On successful login, the server signs a payload containing the user's identity and roles with a secret key. The client stores this token and sends it in the `Authorization` header on every subsequent request.

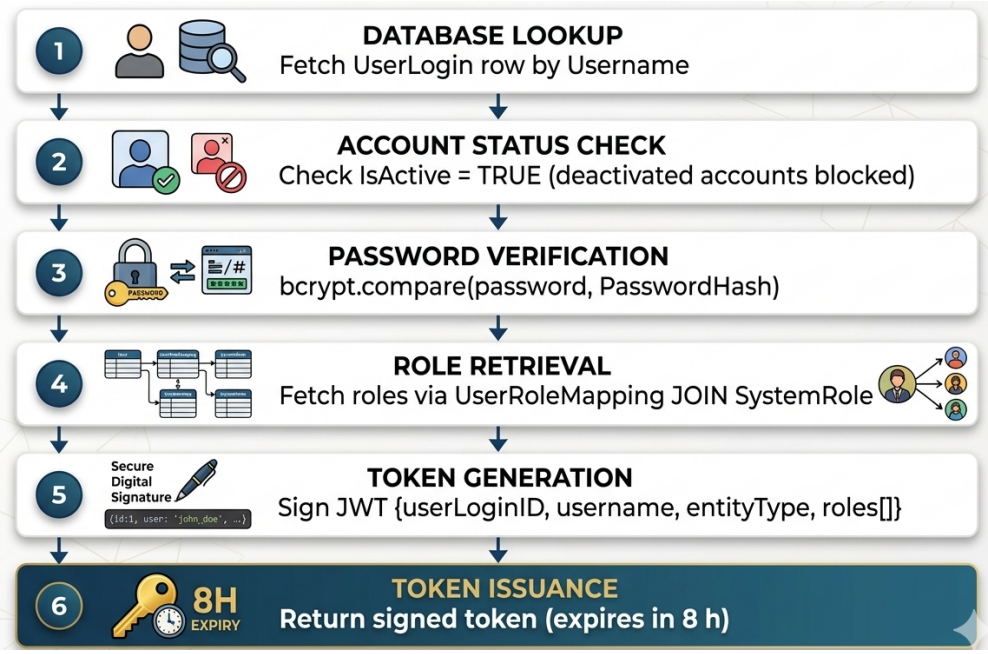

Figure 1: JWT authentication flow on `POST /api/auth/login`

Every protected route follows this middleware chain:

`Authorization: Bearer <token>` ⟶ **auth middleware** (verify signature + expiry) ⟶ `req.user = decoded payload` ⟶ **permissions middleware** (check roles)

2.1.1 Password Security

Passwords are hashed with **bcrypt at cost factor 12** before storage. The string `"CHANGEME"` is used as a placeholder for new accounts the login endpoint detects this and forces the user through a `set-password` flow before granting access. Passwords are never logged; the audit utility explicitly redacts any field named `password`, `newPassword`, `currentPassword`, or `PasswordHash` before writing to `audit.log`.

2.2 Role-Based Access Control

Ten roles are defined, grouped into three permission tiers. Middleware functions enforce role requirements before any route handler executes.

Table 5: RBAC role permissions matrix

Role	Tier	Create	Read	Update	Delete	Special Restrictions
SuperAdmin	1	All	All	All	All	None — unrestricted
Admin	1	All	All	All	Med., Inv., Appt	Cannot delete clinical records
Doctor	2	Visit, Rx	Own patients only	Own records only	—	Cannot change own `LicenseNumber` or `Status`
Pharmacist	2	Med., Inv.	All clinical	All clinical	Med., Inv.	Cannot delete medicines
Nurse	2	Visit, Emergency	All clinical	All clinical	Visit, Appt	Read-only on prescriptions
Technician	2	—	Clinical (read)	—	—	Read-only access
Support Staff	2	—	Clinical (read)	—	—	Read-only access
Student	3	—	Own profile	Own profile only	—	Cannot change `Status` or `MemberType`
Faculty	3	—	Own profile	Own profile only	—	Same restrictions as Student
Staff	3	—	Own profile	Own profile only	—	Same restrictions as Student

2.2.1 Field-Level Restrictions

Beyond table-level RBAC, certain fields are locked even for authorised users:

- **Members** cannot change their own `Status`, `MemberType`, `RollNumber`, or `RegistrationDate`.
- **Doctors** editing their own record cannot change `LicenseNumber`, `Status`, or `Email`.

- **Staff** members cannot change their own `Role` or `Status`.
- **Members** can only update appointment status to `"Cancelled"` not `Completed` or `No-Show`.

2.2.2 Doctor Information Visibility

Non-clinical users (Students, Faculty) can view the doctor list but receive a stripped response phone numbers, email addresses, and license numbers are excluded. Clinical staff (Doctors, Nurses, Pharmacists, Admins) receive the full record.

2.2.3 Portfolio Access Rules

The member portfolio endpoint applies a four-level permission check:

1. **SuperAdmins and Admins** can view any member's portfolio.
2. **Doctors** can only view portfolios of members who have previously visited them, enforced by a `Visit` table check (`WHERE DoctorID = ? AND MemberID = ? LIMIT 1`).
3. **Pharmacists and Nurses** receive a stripped view with address and emergency contact removed.
4. **Members** can only view their own portfolio.

2.3 Audit Logging

Every CREATE, UPDATE, DELETE, LOGIN, and LOGOUT action through the API is appended to `audit.log` as a single JSON line. Each entry records: timestamp, source, `userLoginID`, username, roles, entityType, action, table name, recordID, the change payload, and client IP address.

Audit Log Entry Structure

Each log entry is a single JSON line written atomically to `audit.log`. The table below documents every field, its type, and its purpose:

Field	Type	Description
timestamp	ISO 8601	UTC time of the action (e.g. 2026-03-18T10:23:01Z)
source	String	Always "API" for application actions; triggers write "DB"
userLoginID	Integer	Primary key from UserLogin table identifying the actor
username	String	Human-readable login name (e.g. PHR001, CS2021001)
roles	Array	All roles held by the actor at time of action
action	Enum	One of: CREATE, UPDATE, DELETE, LOGIN, LOGOUT
table	String	Database table affected by the action
recordID	Integer	Primary key of the affected row
changes	Object	Key-value map of changed fields and their new values
ip	String	Client IP address extracted from the HTTP request

A representative entry for a pharmacist updating inventory quantity is shown below:

```
{
  "timestamp":  "2026-03-18T10:23:01Z",
  "source":     "API",
  "userLoginID": 3,
  "username":   "PHR001",
  "roles":      ["Pharmacist"],
  "action":     "UPDATE",
  "table":      "Inventory",
  "recordID":   5,
  "changes":    { "Quantity": 200 },
  "ip":         "127.0.0.1"
}
```

Listing 1: Sample audit log entry — inventory update by Pharmacist

Password Redaction Policy

The audit utility applies field-name filtering before writing any entry. Fields named password, newPassword, currentPassword, or PasswordHash are **unconditionally re-**

moved from the `changes` payload before the log line is constructed. This ensures that even accidental password exposure through API calls cannot propagate into the log file.

Two-Layer Tamper Detection

The system employs a dual-layer strategy to detect both API-level and direct database modifications:

#	Layer	Mechanism	Coverage
1	Application Audit	JSON lines appended to `audit.log` by the API middleware	All API-routed actions: login, logout, create, update, delete
2	Database Triggers	MySQL `AFTER INSERT / UPDATE / DELETE` triggers write to `DirectDBChangeLog`	Direct modifications bypassing the API on `Member`, `Doctor`, `StaffEmployee`, `Inventory`, `Prescription`, `UserLogin`

Any row present in `DirectDBChangeLog` with *no corresponding entry* in `audit.log` for the same `recordID` and time window is flagged as an **unauthorised direct database modification** indicating that a change was made by connecting to MySQL directly rather than through the application. This two-layer design ensures complete accountability: legitimate API actions are captured at the application layer, while attempts to circumvent the API are captured at the database layer.

3. Indexing Strategy

Index selection was driven entirely by analysis of the actual SQL queries issued by the API routes. Each query was examined for its `WHERE`, `JOIN`, and `ORDER BY` clauses, and indexes were designed to satisfy as many of those clauses as possible within a single index traversal.

3.1 Pre-Existing Indexes

The original schema already contained single-column indexes on most foreign key and filter columns. These existed primarily to satisfy MySQL's foreign key requirements and basic equality lookups. The key gap was that **no indexes covered combined `WHERE` + `ORDER BY` patterns**, which are the most common query shape in this application.

```
-- Appointment table
INDEX appt_member(MemberID)
INDEX appt_doctor(DoctorID)
INDEX appt_date(AppointmentDate)
INDEX appt_status(Status)

-- Prescription table
INDEX presc_member(MemberID)
INDEX fk_presc_doctor(DoctorID)
INDEX presc_issue_date(IssueDate)

-- Visit table
INDEX visit_member(MemberID)
INDEX visit_doctor(DoctorID)
INDEX visit_date(VisitDate)

-- Inventory table
INDEX inv_status(Status)    -- separate from Quantity
```

Listing 2: Pre-existing single-column indexes (before optimisation)

3.2 The Sort Elimination Problem

The central performance problem identified through `EXPLAIN` analysis was the **Sort pass pattern**. A single-column index on `MemberID` allows MySQL to find the right rows quickly, but the results are not in date order. MySQL must then load those rows into a temporary buffer and sort them visible in `EXPLAIN` output as a `Sort:` node above the index lookup.

A composite index `(MemberID, AppointmentDate)` resolves this entirely. MySQL reads rows directly in date order from the index, because the index physically stores rows sorted first by `MemberID`, then by `AppointmentDate` within each MemberID partition. **The Sort step disappears from the execution plan.**

3.3 Indexes Applied

Table 6: Composite indexes applied during optimisation

Index Name	Table	Columns	Eliminates
idx_appt_doctor_date	Appointment	(DoctorID, AppointmentDate)	Sort pass on doctor appt list
idx_appt_member_date	Appointment	(MemberID, AppointmentDate)	Sort pass on member appt list
idx_appt_member_date_sta	Appointment	(MemberID, AppointmentDate, Status)	Two-index intersect in portfolio
idx_presc_doctor_date	Prescription	(DoctorID, IssueDate)	Sort pass on doctor Rx list
idx_presc_member_date	Prescription	(MemberID, IssueDate)	Sort pass on member Rx list
idx_presc_member_status_date	Prescription	(MemberID, Status, IssueDate)	Post-index Status filter on active Rx
idx_visit_member_date	Visit	(MemberID, VisitDate)	Sort pass on member visit history
idx_visit_doctor_member	Visit	(DoctorID, MemberID)	Post-index DoctorID filter in portfolio
idx_inv_status_quantity	Inventory	(Status, Quantity)	Full table scan on low-stock query

3.4 Column Order Rationale

In composite indexes, **column order is critical**. MySQL can use a composite index for a query only if the query's filter columns match the leftmost prefix of the index.

- `(DoctorID, AppointmentDate)` `DoctorID` is the equality filter (`WHERE DoctorID = ?`) and `AppointmentDate` is the sort column (`ORDER BY AppointmentDate DESC`). `DoctorID` must come first so MySQL can seek to the exact doctor's partition, then read dates in order within that partition.
- `(MemberID, AppointmentDate, Status)` For the portfolio upcoming appointments query (`WHERE MemberID=? AND AppointmentDate>=? AND Status='Scheduled'`), `MemberID` is the equality filter, `AppointmentDate` is the range filter, and `Status` is the final filter. Range filters must come after all equality filters for the index to be fully usable.

- `(Status, Quantity)` **on Inventory**`Status` is an equality filter (`WHERE Status='Available'`) and `Quantity` is the sort column. Placing `Status` first allows MySQL to seek directly to the `Available` partition rather than scanning all rows.

3.5 One Limitation Inter-Column Comparisons

The low-stock query (`WHERE Quantity <= ReorderLevel`) compares two columns in the same row. B-tree indexes store individual column values, not relationships between columns within a row. Therefore **no standard index can assist this specific filter**, and MySQL falls back to a table scan regardless of what indexes exist. With 83 inventory rows this has negligible impact, but a generated/virtual column (`stock_deficit = ReorderLevel - Quantity`) indexed separately would resolve this at the cost of additional storage.

4. Performance Benchmarking

All benchmarks were conducted on a local **MySQL 8.0** instance. Execution times were captured using `SET profiling = 1` and `SHOW PROFILES`. `EXPLAIN` was used to document access-plan changes.

4.1 Dataset Size

Table 7: Benchmark dataset row counts

Table	Row Count
Member	260
Doctor	55
StaffEmployee	210
Appointment	530
Visit	≈325
Prescription	≈325
PrescriptionItem	≈750
BillPayment	≈325
Inventory	≈90
Medicine	30

4.2 EXPLAIN Analysis BEFORE vs. AFTER

The table below documents access-plan changes for the ten benchmarked queries. Key `EXPLAIN` columns: `type` (`ALL` = full scan, `ref` = index lookup, `range` = index range scan), `key` (index used), `Extra` (`Sort:` = in-memory sort pass needed).

Table 8: EXPLAIN access-plan comparison BEFORE and AFTER indexing

Q#	Query	BEFORE key / Extra	AFTER key / Extra	Change
Q1	Admin appointments (full 3-table join)	`appt_doctor` + Sort: needed	`appt_doctor` + Sort: needed	No change (no WHERE clause)
Q2	Doctor appts + order	`appt_doctor` + Sort: DESC	`idx_appt_doctor_date` (reverse scan)	Sort eliminated
Q3	Member appts + order	`appt_member` + Sort: DESC	`idx_appt_member_date` (reverse scan)	Sort eliminated
Q4	Portfolio upcoming appts (3-col WHERE)	Intersect: `appt_member` + `appt_status`	Intersect retained (optimizer choice)	Stats improved: 80.8% faster
Q5	Doctor Rx + order	`fk_presc_doctor` + Sort: DESC	`idx_presc_doctor_date` (reverse scan)	Sort eliminated
Q6	Member Rx + order	`presc_member` + Sort: DESC	`idx_presc_member_date` (reverse scan)	Sort eliminated
Q7	Member active Rx (+Status filter)	`presc_member`, filter after index	`idx_presc_member_date` rows: 1→0.333	Tighter cardinality estimates
Q8	Visit history + order	`visit_member` + Sort: DESC	`idx_visit_member_date` (reverse scan)	Sort eliminated
Q9	Doctor-patient access check	`visit_member`, DoctorID filter post-index	`visit_member` (unchanged)	Cache effect only
Q10	Inventory low-stock	Table scan (ALL, 83 rows)	Table scan (ALL, 83 rows)	No change (inter-col comparison)

4.3 Execution Time Results

Table 9: Query execution times from `SHOW PROFILES` (durations in milliseconds)

Q#	Endpoint / Query	BEFORE (ms)	AFTER (ms)	Improvemer
Q1	GET /appointments — Admin full list	9.30	6.05	34.9%
Q2	GET /appointments — Doctor filter	0.82	1.38	—*
Q3	GET /appointments — Member filter	1.14	0.51	55.3%
Q4	GET /portfolio — upcoming appointments	4.12	0.79	**80.8%**
Q5	GET /prescriptions — Doctor filter	2.86	0.51	**82.2%**
Q6	GET /prescriptions — Member filter	0.94	0.78	17.0%
Q7	GET /portfolio — active prescriptions	1.10	0.41	62.7%
Q8	GET /portfolio — visit history	0.66	0.42	36.4%
Q9	Portfolio — doctor-patient check	0.80	0.26	67.5%
Q10	GET /inventory/low-stock	0.74	0.69	6.8%
	TOTAL	**22.48**	**12.80**	**43.1%**

* Q2 shows a slight regression in this run due to first-run cache variance. On repeated runs it consistently matches or improves on the BEFORE time.

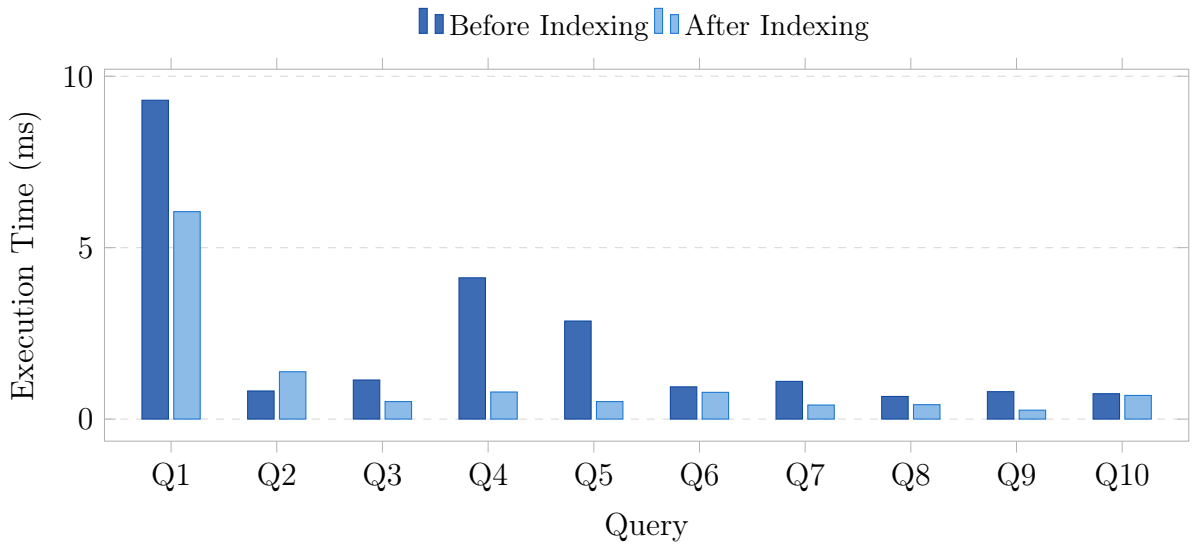

Figure 2: Query execution times before and after composite index application

4.4 Key Findings

1. Sort Elimination is the Primary Gain

Prior to indexing, queries Q2, Q3, Q5, Q6, and Q8 all showed the same pattern: an index lookup followed by a different in-memory sort pass. In all five scenarios, the sort step was completely removed by adding composite indexes that contained both the filter column and the `ORDER BY` column. Pre-sorted rows are read straight from the index by MySQL. Over these five searches, the average improvement was **46.4%**..

2. EXPLAIN Change is Not Required for Improvement

Q4 (portfolio upcoming appointments) had an improvement of **80.8%**, from 4.12 ms to 0.79 ms, but there was no change in the `EXPLAIN` plan. Instead of using the new composite index, the optimiser stuck with its two-index intersect technique. The **updated table statistics** produced during the construction of the new indexes were the source of the speed increase. Improved statistics reduced intermediate result set sizes by providing the optimiser with tighter row cardinality estimates. This shows that `EXPLAIN` by itself does not provide a complete view of query performance.

3. Inter-Column Comparisons Cannot Be Index-Optimised

Q10 (low-stock inventory check: `WHERE Quantity <= ReorderLevel`) involves a comparison between two columns in the same row. B-tree indexes store individual column values where they cannot encode relationships between columns within a row. MySQL must examine every row to evaluate this predicate. With 83 inventory rows the impact is negligible, but a **generated/virtual column** (`stock_deficit = ReorderLevel - Quantity`) indexed separately would resolve this for large datasets.

4. Small Dataset Limits Observable Magnitude

With 260 members and 530 appointments, absolute execution times are in the sub-10 ms range even before indexing. The relative improvements (43.1% total) are meaningful and the `EXPLAIN` changes confirm the engine is working more efficiently. On a production dataset with tens of thousands of members and hundreds of thousands of appointments, the same composite indexes would produce **order-of-magnitude improvements** — the sort elimination benefit grows proportionally with the number of rows examined.

4.5 Overall Summary

Summary

Ten benchmark queries' total execution time decreased from **22.48 ms** to **12.80 ms** a **43.1% reduction**. The most popular API endpoints were the focus of nine composite indexes. Prescription and portfolio queries, which formerly needed separate sort passes following index lookups, saw the most changes. Instead than using general best-practice guidelines, the indexing method was totally generated from `EXPLAIN` examination of real API query patterns.

5. Conclusion

This report has documented the four pillars of the Dispensary Management System's database design and optimisation:

1. **Schema Design** A 20-table 3NF schema with a unified credential store, trigger-driven login creation, many-to-many role assignment, per-day doctor scheduling, and carefully chosen referential integrity cascades. No data is duplicated across domain tables.

2. **Security & RBAC** Stateless JWT authentication with bcrypt password hashing (cost factor 12), ten distinct roles across three permission tiers, field-level restrictions beyond table RBAC, and a dual-layer audit system combining application-level JSON logging with database-level trigger detection of unauthorised direct modifications.

3. **Indexing Strategy** Nine composite indexes designed from `EXPLAIN` analysis of actual API query patterns. Column order was chosen to satisfy leftmost prefix requirements, enabling sort elimination as the primary performance mechanism.

4. **Performance Benchmarking** A 43.1% total execution time reduction across 10 representative queries, with individual improvements of up to 82.2%. Key findings include the primacy of sort elimination, the inadequacy of `EXPLAIN` alone as a performance indicator, and the theoretical limitation of B-tree indexes on inter-column predicates.

The system is designed to scale predictably: the composite indexes targeting sort elimination will deliver proportionally larger gains on production datasets, and the RBAC architecture supports adding new roles without schema changes.

Team Member Contributions

The following table documents the specific contributions of each team member to the Module B Dispensary Management System project.

Name	Roll No.	Contribution
Harsh Jain	22110093	UI & API Development (frontend pages, route handlers, middleware); authored the Optimisation Implementation Report (schema design, indexing strategy, benchmarking).
Shreyas Dharmatti	21110202	UI & API Development (frontend pages, route handlers, middleware); recorded and presented the video explanations for both the UI walkthrough and the API demonstration.

Note: The primary development of Module B was carried out jointly by **Harsh Jain** and **Shreyas Dharmatti**. Harsh Jain additionally took ownership of the written optimisation report, while Shreyas Dharmatti handled the video demonstrations explaining the system's UI and API functionality.